Designmönster (design patterns)

En introduktion samt mönstret Strategy

Vad är ett designmönster?



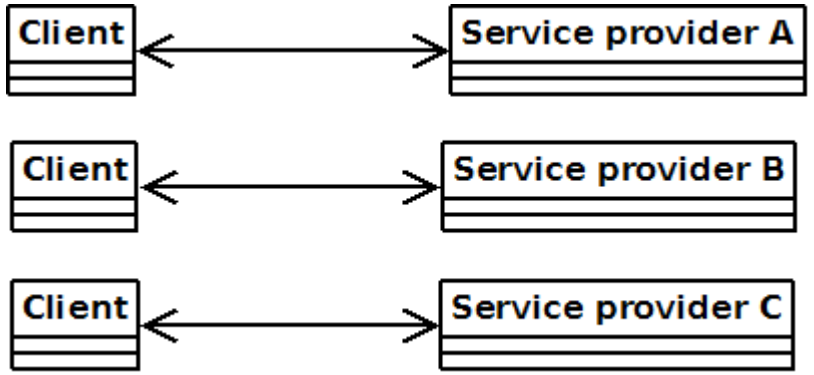


- Ett designmönster är en (namngiven) lösning på ett i en viss omgivning ofta återkommande problem.
- I mjukvaru-världen: En uppsättning klasser med givna relationer och operationer som löser ett återkommande problem.

Först: Önskvärt i OO kod

- Klasser ska ha hög kohesion - en och endast en roll
- Låg koppling mellan moduler/klasser – beroendet mellan klasser ska vara minimalt, en ändring i en klass ska ha så liten påverkan som möjligt på omgivande klasser
- Programmera mot gränssnitt, inte mot implementationer

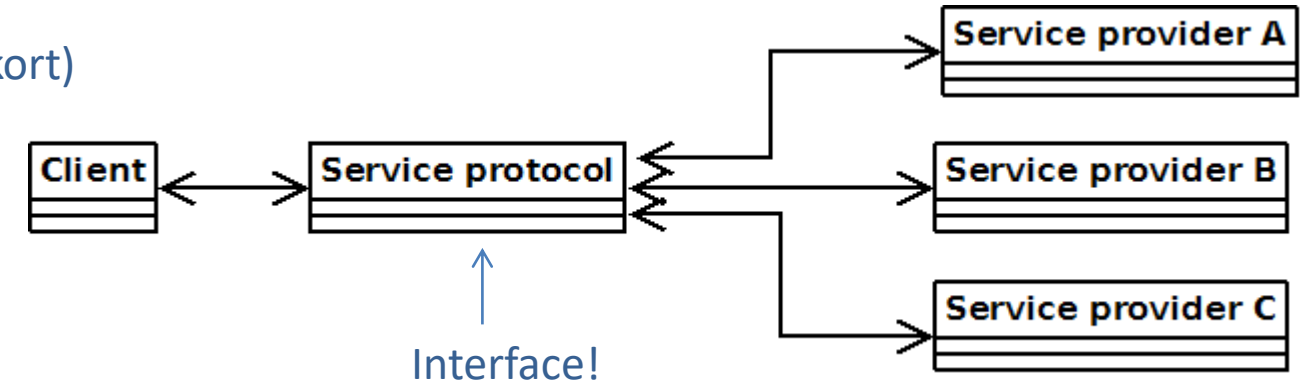
Abstract coupling - programmera mot interface



Här behöver varje typ av samtalstjänst en speciell mobilmodell!

T.ex. mobil

T.ex. samtalstjänst (via simkort)



Klienterna behöver nu inte känna till eller anpassa sig efter specifika tjänsteleverantörer - leverantören blir utbytbar

Abstract coupling

- Programmera mot interface, inte mot implementationer
  - Klasser och pakets publika gränssnitt (och allt övrigt privat)
  - Ett `interface` med alternativa, för klienten okända(!), underliggande implementationer
- Poänger:
  - Klienten behöver (bör) inte känna till detaljer om implementationen, utan endast gränssnittet
  - Låg koppling – kan ändra i implementationen utan att det påverkar klienterna
- List<String> list = new ArrayList<>() eller new LinkedList<>()

Första designmönstret: Ett enda objekt, tillgängligt överallt

- System för kölappar …



- … om det finns flera automater för samma kö …
- … *endast ett objekt* som genererar nästa nummer i systemet
- QueueTicketDispenser, getNextNumber()

- Andra exempel då vi vill garantera ett enda objekt, tillgängligt överallt
  - resurskrävande objekt
  - en enda resurs ska hanteras, t.ex.
    - koppling till databas
    - window manager
    - timer för hela systemet
    - …
- Naiv lösning:

```
/** Create only one instance of this class */
public class QueueTicketDispenser { …
```

Ett enda objekt, tillgängligt överallt – första försöket

Objekträknare (dålig lösning)

```
public class QueueTicketDispenser {
    private static int objectCount = 0;
    private int nr;

    public QueueTicketDispenser() throws Exception {
      if(objectCount > 0) throw new Exception();
      nr = 1;
      objectCount++;
    }

    public int getNextNumber() {
      return nr++;
    }
}
```

Ett enda objekt, tillgängligt överallt – första försöket

- Problem med lösningen (objekträknare)?
  - Om vi försöker skapa fler objekt upptäcks detta först vid exekvering (Exception, run-time)
  - Skulle referensen försvinna kan vi inte återskapa objektet, eller inte längre komma åt det objekt som finns

Ett enda objekt, tillgängligt överallt

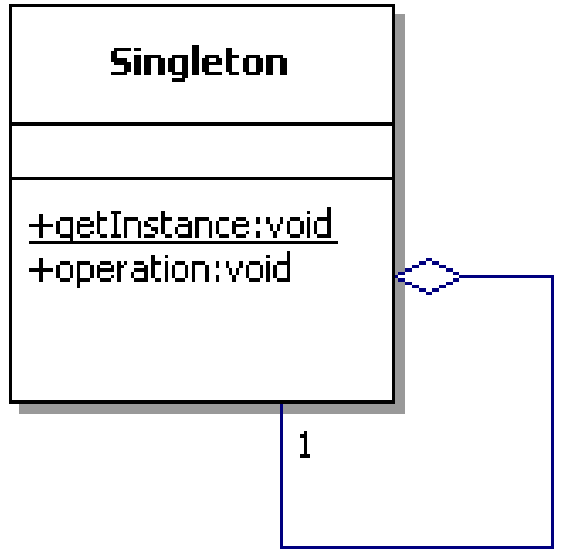
- Bästa lösningen(?), dvs. designmönstret: Gör konstruktorn privat(!)
  - Enda sättet att få tillgång till objektet via klassmetod (static)

```
public class QueueTicketDispenser {
    private int nr;
    private static QueueTicketDispenser theObject;

    public static QueueTicketDispenser getInstance() {
         if(theObject == null) theObject = new QueueTicketDispenser();
         return theObject;
    }
    private QueueTicketDispenser() {
         nr = 1;
    }
    public int getNextNumber() {
         return nr++;
    }
}
```

Ett enda objekt, tillgängligt överallt - Singleton

- Designmönstret Singleton
  - Endast ett objekt, tillgängligt överallt
  - Objektet skapas först när det behövs
  - Lösningen ej trådsäker … … går dock att lösa (med synchronized)



- Hackerns favoritmönster. Risk för sjukdomen ”Singeltonit”.

Typer av designmönster

- Skapelsemönster
  - Singleton, diverse Factory-mönster, Iterator, Prototype, …
  - Gemensamt för dessa: Vi vill undvika ”new” – exakt vilken (sub-) typ det skapade objektet ska ha avgörs internt
- Strukturmönster
  - Facade, Model-View-Controller, Composite, Proxy, …
- Beteendemönster
  - Strategy, Template method, Command, …

Problem – urval enligt olika kriterier

- Antag att vi i en klass med en samling objekt vill kunna söka objekt som matchar ett kriterium …
- … men det finns många kriterier, och nya kan tillkomma
- Vi kan naturligtvis skriva en uppsättning, nästan identiska sökmetoder, en för varje kriterium …
- … men vi ska så långt som möjligt undvika upprepad kod.
  - Vi vill också slippa uppdatera klassen när vi vill lägga till nya sökkriterier

```
public class MusiciansList {
    private ArrayList<Musician> musicians;
    …

    // match artist name
    public List<Musician> search(String artistName) {
        List<Musician> searchResult = new ArrayList<>();
        for(Musician musician: musicians) {
            if(musician.getArtistName().contains(artistName)) {
                searchResult.add(musician);
            }
        }
        return searchResult;
    }

    // match artist genre
    public List<Musician> search(Genre genre) {
        List<Musician> searchResult = new ArrayList<>();
        for(Musician musician: musicians) {
            if(musician.getGenres().get(0).equals(genre) {
                searchResult.add(musician);
            }
        }
        return searchResult;
    }

    // match artist …
```

Designmönstret Strategy

- Vi kan istället definiera ett interface, som deklarerar en metod för det som skiljer sökmetoderna åt …
- … delstrategin, i vårt fall sökkriteriet
- Vi behöver nu endast skriva en enda sökmetod, som tar som argument en implementation av interfacet för strategin/kriteriet

```
public interface IMusicianMatcher {

    /**
     * Return true if the argument matches the criteria in the implementatio
     * this interface.
     * @param musician the object to check for the criteria
     * @return true if the argument matches the criteria
     */
    boolean match(Musician musician);
}

------

public class MusiciansList {
    private ArrayList<Musician> musicians;
    …

    public List<Musician> search(IMusicianMatcher matcher) {
        List<Musician> searchResult = new ArrayList<>();
        // search
        for(Musician musician: musicians) {
            if(matcher.match(musician)) {
                searchResult.add(musician);
            }
        }
        return searchResult;
    }
}
```

Designmönstret Strategy

- Nu kan vi skriva olika implementationer av interfacet …
- … och anropa sökmetoden med instanser av dessa
- Om vi behöver lägga till ett nytt sökkriterium skriver vi en ny implementation av strategi-interfacet
  - Sökmetoden behöver inte uppdateras

```
public class ArtistNameMatcher implements IMusicianMatcher {

    private final String searchForArtistName;

    public ArtistNameMatcher(String searchForArtistName) {
        this.searchForArtistName = searchForArtistName;
    }

    @Override
    public boolean match(Musician musician) {
        return musician.getArtistName().contains(searchForArtistName);
    }
}

public class GenreMatcher implements IMusicianMatcher {
    …
}

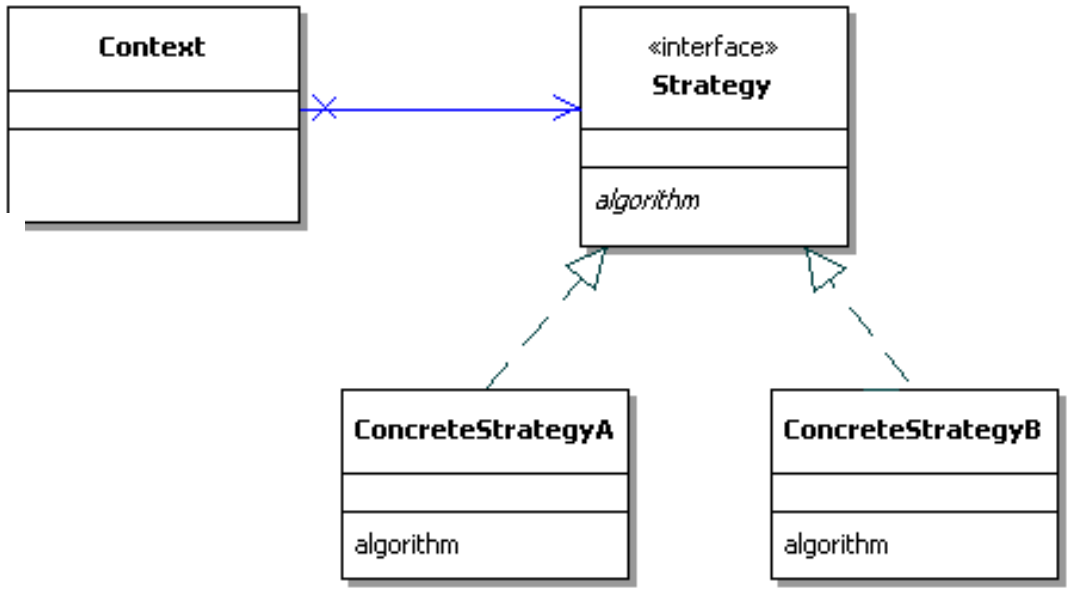
------

List<Musician> genreResult = musicianList.search(
                                        new GenreMatcher(Genre.SOUL));
```

Dersignmönstret Strategy - att kapsla in beteende

- Syfte: Att kapsla in beteende att användas som strategi/delmoment i andra algoritmer, (del-) strategier kan bytas ut dynamiskt
- Ersätter villkorade specialfall

```
void context(Strategy strategy) {
    …;
    strategy.algorithm();
    …;
}
-----
context(new ConcreteStrategyA());
…;
```



- Polymorfism – vi skickar metod-implementationen via en interface-referens

Strategy vid rangordning och sortering

- Vi önskar använda en och samma sorteringsalgoritm, men göra kriterierna vi använder, för att jämföra objekten, utbytbara
  - Vi vill kunna ha flera olika kriterier för rangordning för en och samma typ
- Vi behöver ett interface som deklarerar en metod för rangordningen
  - Metoden behöver två argument, referenser till de objekt som ska rangordnas
- Interfacet java.util.Comparator:

```
public interface Comparator<T> {

    public abstract int compare(T obj1,T obj2);
    …
}
```

  - Om obj1 ”mindre än” obj2, returnera negativt,
    om obj1 identiskt med obj2, returnera 0,
    om obj1 ”större än” obj 2, returnera positivt

Strategy med Comparator

- Vi vill göra det möjligt att rangordna objekt av typen Musician enligt flera olika kriterier
- Vi vill kunna lägga till nya kriterier för rangordning utan att behöva ändra i klassen Musician
  - Implementationerna av Comparator skrivs som separata klasser

```
public class Musician {

    private final String name;
    private final String artistName;
    private final ArrayList<Genre> genres;
    …
}
```

```
public class ArtistNameComparator implements Comparator<Musician> {

    @Override
    public int compare(Musician m1, Musician m2) {
        return m1.getArtistName().compareTo(m2.getArtistName());
    }
}
```

```
public class GenreComparator implements Comparator<Musician> {

    @Override
    public int compare(Musician m1, Musician m2) {
        Genre g1 = m1.getGenres().get(0);
        Genre g2 = m2.getGenres().get(0);
        return g1.ordinal() - g2.ordinal(); // the order in which the values are declared in enum
    }
}
```

Strategy vid rangordning och sortering

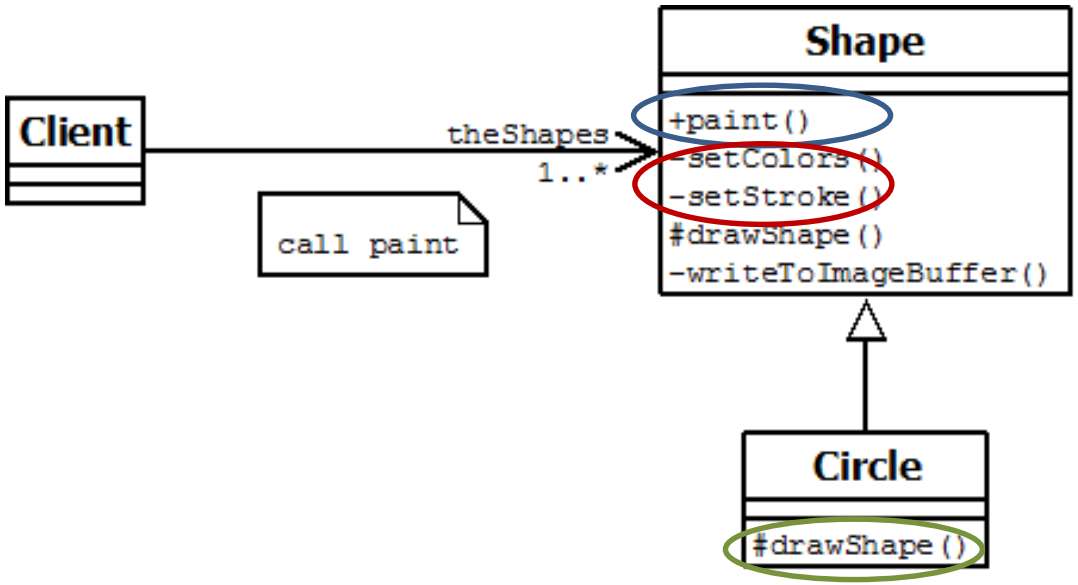
- Sorteringsalgoritm som använder Comparator
- public static <T> void sort (List<T> list, Comparator<T> comparator) {

```
    …
    for( … ) {
        for( … ) {
            if(comparator.compare(list.get(i),list.get(j)) < 0) {
            // Swap…
        }
        …
    …
}
```

- Och, naturligtvis finns redan java.util.Collections.sort(List<T>, Comparator<T>)

Template method

- Problem: Vi har, i en *arvshierarki*, en publik metod som ska utföras på olika sätt i olika subklasser (override), men …
- … där delar av implementationen ska vara gemensam i alla subklasser
- Lösning: Dela upp koden i separata metoder, som alla anropas av en metod i superklassen, deklarerad final
- Kodstycken som ska vara identiska i alla subklasser implementeras, privata och/eller final, i superklassen
- Kodstycken som kan se olika ut i olika subklasser deklareras i metoder som kan omdefinieras, men är skyddade (protected)



Template method

```
public abstract class Shape {

    public final void paint(…) {
        setColors();
        setStroke();
        drawShape(…);
        writeToImageBuffer();
    }

    private void setColors() { // ... }
    private void setStroke() { // ... }
    private void writeToImageBuffer() { // ... }

    // draw, using g. Override in subklasses
    protected abstract void
        drawShape(GraphicsContext g);

    …
```

final: kan inte omdefinieras i subklasser

```
public class Circle extends Shape {

    @Override
    proteced void drawShape(GraphicsContext g) {
        // draw a circle, using g
        …
        …
    }

    …
}
```

Create an object:

```
Circle c = new Circle(…);
c.paint(…); // call drawShape in Circle, polymorph
```